



**Implementación de una infraestructura virtualizada
para la monitorización eólica mediante
contenedores Docker**

Curso: 2º Sistemas Microinformáticos y Redes (SMR)

Fecha:5/04/2026

Alumno: Alba Cortés Sánchez

Tutor TFG: Raúl Albiol Salguero

PROMETEO

DEDICATORIA:

Quiero dedicar el proyecto “EOLO” por todo el tiempo pegada en la pantalla, entre tanto agobio y horas. Este proyecto tiene mucho esfuerzo y más de un momento de querer mandarlo todo a paseo por los errores que salían, pero al final ha salido. Gracias por estar ahí.

AGRADECIMIENTOS:

Quiero agradecer a todas esas personitas que han formado parte de este proceso y me han ayudado a llegar hasta aquí.

En primer lugar, a mi familia, por vuestro apoyo y por la paciencia que habéis tenido conmigo durante todos estos meses. Sin vuestra confianza en mí, esto habría sido más complicado.

A mis amigos, por estar siempre ahí. Gracias por las risas y por las charlas de desconexión. Vuestro tiempo con vosotros ha sido fundamental.

A todos vosotros, gracias por vuestro tiempo y vuestro apoyo para sacar este proyecto adelante.

ÍNDICE:

JUSTIFICACIÓN DEL PROYECTO:

1.INTRODUCCIÓN:

2.OBJETIVO:

3.DESCRIPCIÓN:

- 3.1.Preparación del Entorno Virtualizado
- 3.2. Preparación y Actualización del Sistema Operativo
- 3.3. Instalación de la Plataforma de Contenedores
- 3.4. Verificación de la Instalación
- 3.5. Gestión del Servicio
- 3.6. Preparación del Espacio
- 3.7. Estructuración del Microservicio "Simulador"
- 3.8. Creación del Proyecto
- 3.9. Generación de Componentes del Microservicio
- 3.10. Verificación de la Estructura de Archivos
- 3.11. Desarrollo del Script de Simulación (simulador.py)
- 3.12. Construcción de la Imagen con Dockerfile
- 3.13. Orquestación del Sistema con Docker Compose
- 3.14. Despliegue de la infraestructura (docker-compose up)
- 3.15. Configuración Inicial de InfluxDB (Interfaz Web)
- 3.16. Finalización del Setup y confirmación del entorno
- 3.17. Panel de control principal de InfluxDB
- 3.18. Obtención del API Token para la conexión con Python
- 3.19. Detalle de privilegios y copiado del Token
- 3.20. Vinculación final del Script con el Token de Seguridad
- 3.21. Verificación de datos (Data Explorer)
- 3.22. Configuración de Grafana y Visualización de Datos
- 3.23. Interfaz principal de Grafana
- 3.24. Configuración detallada de la conexión HTTP y Autenticación
- 3.25. Creación y edición de paneles de visualización
- 3.26. Implementación de consultas en lenguaje Flux
- 3.27. Análisis de los datos monitorizados (Métricas de Python)

4.TECNOLOGÍA:

5.METODOLOGÍA:

6.TRABAJOS FUTUROS:

7.CONCLUSIÓN:

8.REFERENCIAS:

ABSTRACT:

English version:

The project “EOLO” details the design and implementation of a professional telemetry and monitoring system for a wind turbine, developed within a Linux server environment. The architecture is based on a microservices model: a Python script functions as a simulated sensor node, streaming data to a Docker-containerized core. The stack features InfluxDB for time-series storage and Grafana for dynamic visualization. The project demonstrates key SMR competencies, including advanced service administration, container network management, and authentication protocols in production environments.

Versión Español:

El proyecto “EOLO” describe el diseño y la implementación de un sistema profesional de telemetría y monitorización para un aerogenerador, desarrollado íntegramente en un entorno de servidor Linux. La arquitectura sigue un modelo de microservicios: un script de Python actúa como nodo sensor simulado, enviando datos a un núcleo de sistema contenedorizado con Docker. Se utiliza InfluxDB para el almacenamiento de series temporales y Grafana para la visualización dinámica. El proyecto destaca competencias de SMR, como la administración de servicios por interfaz de comandos, la gestión de redes de contenedores y la resolución de errores de autenticación en entornos de producción.

JUSTIFICACIÓN DEL PROYECTO:

La motivación de este proyecto parte de la necesidad de modernizar la gestión de infraestructuras críticas mediante soluciones de código abierto. La dependencia de software propietario genera costes elevados y sistemas rígidos. Este proyecto busca demostrar que, mediante la integración de sistemas abiertos, es posible diseñar una alternativa escalable y eficiente que democratice el acceso a la monitorización industrial de alto nivel, alineándose con las tendencias actuales de la Industria 4.0 (**Tanenbaum & Wetherall, 2021**).

Actualmente, el mercado está dominado por sistemas SCADA (*Supervisory Control and Data Acquisition*). Aunque son robustos, su implementación es costosa y su interoperabilidad es limitada (**Tanenbaum & Wetherall, 2021**). El público objetivo comprende pequeñas y medianas empresas (PYMES) del sector energético, centros de formación y también comunidades de energía local.

A diferencia de las herramientas de monitorización de redes tradicionales, este proyecto se enfoca en la visualización de series temporales, utilizando un stack tecnológico que optimiza la experiencia de usuario y la velocidad de respuesta (**Grafana Labs, 2026**).

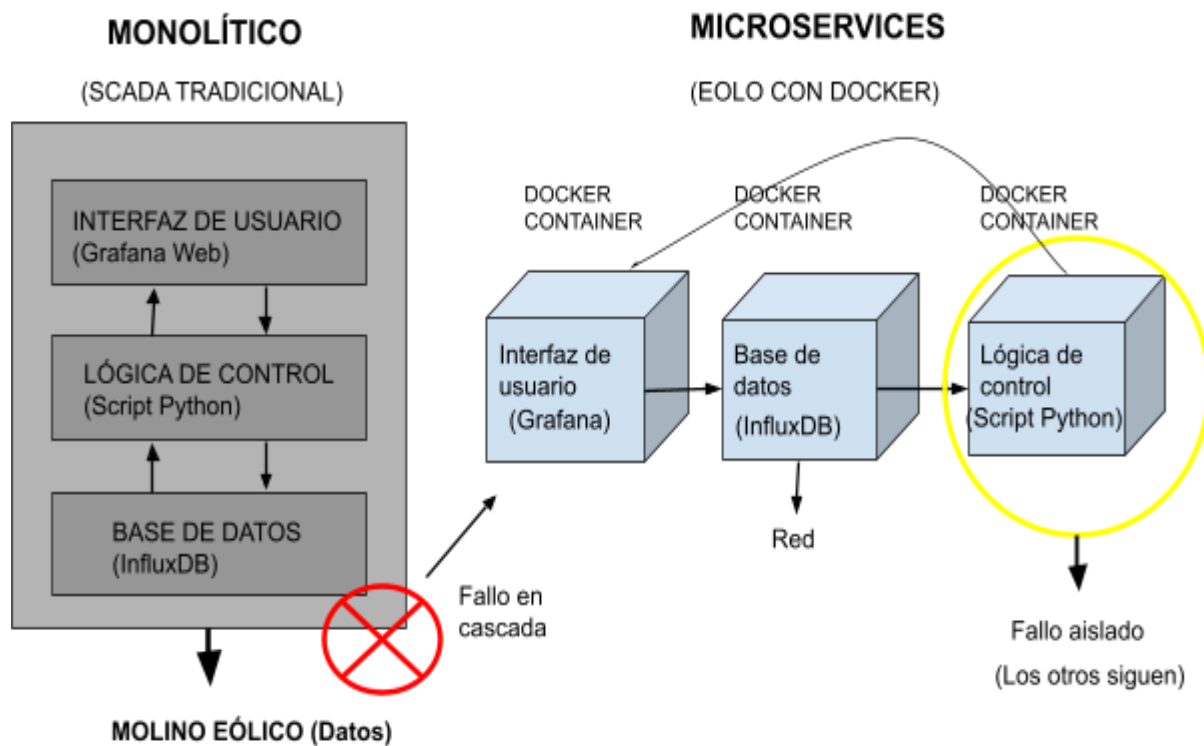
Comparativa técnica:

En primer lugar, frente a la arquitectura monolítica de los sistemas SCADA tradicionales, este proyecto apuesta con Docker. He elegido Docker porque, a diferencia de los sistemas monolíticos tradicionales, permite que cada componente sea independiente (**Docker Inc., 2026**). Esto garantiza que la captura de datos sea robusta y que el sistema sea fácil de mantener y escalar sin costes adicionales.

La tolerancia a fallos es importante: al aislar los servicios, si la interfaz visual de Grafana requiere un reinicio, el motor de la base de datos y la captura de datos del aerogenerador siguen funcionando de manera ininterrumpida.

Figura 1.

Comparativa de arquitectura monolítica frente a microservicios orientada al proyecto EOLO



Nota. Elaboración propia.

Como se puede observar en la comparativa de arquitecturas (Figura 1), el aislamiento de servicios es importante.

-En el modelo Monolítico: El flujo de datos es vertical y dependiente; esto implica que cualquier error en la capa de la Base de Datos o en la Lógica de Control bloquea inmediatamente la Interfaz de Usuario.

En un entorno industrial, esto supondría una pérdida total de visibilidad del aerogenerador ante un fallo menor.

-En el modelo de Microservicios EOLO: La arquitectura desplegada en este proyecto utiliza contenedores Docker para separar funciones:

Simulación: El Script Python funciona de forma independiente, enviando ráfagas de datos a la base de datos sin esperar respuesta de la interfaz, simula el comportamiento físico de un aerogenerador, transformando mediante algoritmos la energía cinética del viento en potencia eléctrica y enviando ráfagas de telemetría sin bloquear el resto del sistema.

Visualización: Grafana realiza consultas a InfluxDB solo cuando es necesario refrescar la visualización, la capacidad para conectar con InfluxDB de forma nativa permite crear alertas en tiempo real, para detectar fallos del aerogenerador antes de que ocurra una avería grave.

Almacenamiento: InfluxDB está optimizada para almacenar telemetría, permitiendo consultas rápidas y una compresión de datos, lo que ahorra espacio en el disco del servidor.

Tolerancia a fallos: Como indica el círculo de "Fallo aislado", si el servicio se detuviera, los servicios de visualización y almacenamiento permanecerán activos, permitiendo consultar los datos mientras se soluciona el incidente.

1.INTRODUCCIÓN:

En la actualidad, la digitalización y el análisis de datos en tiempo real se han convertido en pilares fundamentales para el desarrollo del sector industrial. La industria de las energías renovables demanda sistemas de supervisión constantes que permitan optimizar la producción y garantizar la seguridad de las infraestructuras. Sin embargo, el acceso a estas tecnologías suele estar limitado por los altos costes de implementación y complejidad de los sistemas.

Este proyecto, surge como una respuesta a esta situación. El objetivo es diseñar una infraestructura técnica que sea capaz de capturar, procesar y visualizar datos complejos en tiempo real, utilizando herramientas de código abierto, donde cada componente del sistema está optimizado para cumplir una función específica dentro de una red local segura.

Para lograrlo, se ha diseñado una arquitectura basada en la virtualización de sistemas y la contenedorización de servicios. Utilizando un entorno Linux, se integrarán herramientas como Docker, InfluxDB y Grafana, tecnologías que permiten el aislamiento de procesos y la gestión eficiente de datos temporales (**Docker Inc., 2026; InfluxData, 2026**). Esta infraestructura permitirá que un nodo sensor simule el comportamiento de un aerogenerador y envíe su telemetría a un servidor centralizado mediante protocolos de red (**Tanenbaum & Wetherall, 2021**).

A través de este documento, se detalla el proceso de configuración de redes, la administración de servidores y la gestión de contenedores, demostrando la viabilidad de construir soluciones de nivel industrial con recursos optimizados. Este proyecto no solo constituye un reto personal, sino que actúa como la validación final de las competencias adquiridas en el ciclo de Sistemas Microinformáticos y Redes, aplicadas a un entorno real de innovación y sostenibilidad.

2.OBJETIVO:

R01 - Visualización en Tiempo Real: El sistema debe permitir la monitorización de los datos del aerogenerador a través de una interfaz web accesible.

-R01F01 - Interfaz gráfica de monitorización: Implementación de un entorno visual para el análisis de datos.

-R01F01T01 - Despliegue de Grafana: Configurar el servicio mediante contenedores Docker.

-R01F01T01P01 - Acceder a la URL <http://localhost:3000> y validar la carga del servicio.

-R01F01T02 - Diseño de Dashboard: Configurar un panel con consultas en lenguaje Flux para representar el viento y la potencia.

-R01F01T02P01 - Validación de datos: Visualizar las métricas en el navegador del cliente.

R02 - Almacenamiento de Series Temporales: Los datos deben persistir de forma organizada para permitir las consultas y análisis de rendimiento.

-R02F01 - Base de Datos Especializada: Uso de un motor optimizado para eventos temporales.

-R02F01T01 - Despliegue de InfluxDB: Configurar el contenedor y crear el bucket de datos denominado eolo _data.

-R02F01T01P01 - Estado del servicio: Ejecutar docker ps y confirmar que el contenedor eolo db está en ejecución.

-R02F01T02 - Seguridad y Autenticación: Generar un API Token para permitir la seguridad de los datos externos.

-R02F01T02P01 - Prueba de persistencia: Realizar una consulta desde el Data Explorer de InfluxDB para validar la recepción de registros.

R03 - Simulación de Telemetría: Un programa automatizado debe generar y enviar los datos del molino simulando el comportamiento de sensores reales.

-R03F01 - Nodo Sensor Virtual: Desarrollo de un script de telemetría.

-R03F01T01 - Script Python: Creación de un programa que calcule valores aleatorios de viento y potencia basados en curvas de rendimiento reales.

-R03F01T01P01 - Verificación Local: Ejecutar el script y validar la salida de datos por la terminal del cliente.

-R03F01T02 - Integración API: Configurar la conexión HTTP hacia el host `http://influxdb:8086` usando el Token de autenticación.

-R03F01T02P01 - Validación Extrema: Comprobar en los logs del contenedor `eolo-sim-script` el mensaje de confirmación.

R04 - Infraestructura y Red - Garantizar la comunicación aislada y segura entre el nodo sensor y el servidor de control.

-R04F01 - Segmentación de Red: Configuración de una red privada virtual para los servicios.

-R04F01T02 - Direccionamiento Estático: Asignar IPs fijas en los sistemas Ubuntu para evitar conflictos de resolución de nombres.

-R04F01T02P01 - Auditoría de Red: Verificar la configuración mediante el comando `ip addr` en ambas terminales Linux.

3.DESCRIPCIÓN:

En este apartado se detalla el proceso del sistema **EOLO**, desde la preparación del entorno virtualizado hasta la puesta en marcha de los microservicios de monitorización.

3.1.Preparación del Entorno Virtualizado

Para garantizar un entorno de ejecución aislado, escalable y seguro, se ha optado por una solución de virtualización utilizando Oracle VM VirtualBox.

-Sistema Operativo: Ubuntu 24.04 LTS (Linux). Se eligió esta distribución por su estabilidad y compatibilidad con Docker.

-Recursos de Hardware Virtual:

Procesamiento: 2 CPUs para asegurar la fluidez de los procesos.

Memoria RAM: 3250 MB, optimizados para el consumo de la base de datos de series temporales.

Almacenamiento: Disco 33,91 GB.

-Configuración de Red: Se ha configurado un Adaptador Puente. Permite que la máquina virtual se comporte como un nodo real dentro de la red física, obteniendo una IP del servidor DHCP y facilitando el acceso a los puertos 3000 (Grafana) y 8086 (InfluxDB) desde cualquier dispositivo de la red local.

3.2. Preparación y Actualización del Sistema Operativo

Antes de proceder con la instalación de los servicios de monitorización, se realizó una auditoría de seguridad y actualización de los repositorios de Ubuntu. Este paso es importante para asegurar que todas las dependencias y parches de seguridad estén al día.

-Comando ejecutado: `sudo apt update`

Se ejecutó **sudo apt update** para descargar las listas de paquetes más recientes de los servidores.

```
eolo@eoloalba:~$ sudo apt update
[sudo] password for eolo:
Hit:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:4 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
4 packages can be upgraded. Run 'apt list --upgradable' to see them.
eolo@eoloalba:~$
```

Posteriormente, se lanzó el comando **sudo apt upgrade -y**. El parámetro -y permite automatizar la confirmación de la descarga e instalación.

```
eolo@eoloalba:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages will be upgraded:
  cloud-init python3-software-properties software-properties-common systemd-hwe-hwdb
4 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

-Finalidad: Este procedimiento asegura que las librerías necesarias para Docker y Python 3 estén en su versión estable, evitando errores durante el despliegue del simulador de telemetría.

3.3. Instalación de la Plataforma de Contenedores

Para permitir que los microservicios se ejecuten de forma aislada, se instaló el motor de Docker.

-Comando ejecutado: `sudo apt install docker.io docker-compose -y`

-Justificación: Se utiliza docker.io para la gestión de imágenes y contenedores, y docker-compose para la comunicación.

```
eolo@eoloalba:~$ sudo apt install docker.io docker-compose -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
docker.io is already the newest version (28.2.2-0ubuntu1~24.04.1).
docker-compose is already the newest version (1.29.2-6ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
eolo@eoloalba:~$
```

3.4. Verificación de la Instalación

Tras completar la descarga de los paquetes, es fundamental validar que el motor de contenedores esté operativo y responda correctamente.

-Comando ejecutado: `docker --version`

```
eolo@eoloalba:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
eolo@eoloalba:~$
```

-Finalidad: Confirmar que las versiones instaladas son compatibles con las librerías de InfluxDB y Grafana que utilizaremos en el archivo YAML.

-Resultado: La respuesta de la terminal confirma que está correctamente enlazado en el sistema, permitiendo la ejecución de comandos de administración sin errores.

3.5. Gestión del Servicio

Una vez verificada la instalación, el siguiente paso es garantizar que el sistema de monitorización sea capaz de recuperarse automáticamente ante un reinicio del servidor o un fallo eléctrico.

-Comando ejecutado: `sudo systemctl enable --now docker`

```
eolo@eoloalba:~$ sudo systemctl enable --now docker
[sudo] password for eolo:
eolo@eoloalba:~$
```

-Justificación: Con **enable**, configura Docker para que se inicie automáticamente durante el proceso de arranque del sistema operativo, y con **--now**, activa el servicio de forma inmediata en la sesión actual sin necesidad de reiniciar.

-Resultado: Con esta configuración, será capaz de levantarse solo sin intervención manual del administrador.

3.6. Preparación del Espacio

Con el motor de contenedores configurado y operativo, se procedió a la creación de una estructura de directorios. Este paso es importante en la administración de sistemas para mantener el aislamiento del proyecto y facilitar la gestión de los datos.

-Comando ejecutado: `mkdir EOLO-SMR y cd EOLO-SMR`

```
eolo@eoloalba:~$ mkdir EOLO-SMR
eolo@eoloalba:~$ cd EOLO-SMR
```


-Finalidad: Se crea la carpeta raíz del proyecto denominada EOLO-SMR y se accede a ella. A partir de aquí, todos los archivos de configuración y scripts se centralizan en esta ubicación para garantizar la portabilidad del sistema.

3.7. Estructuración del Microservicio "Simulador"

Para mantener la independencia entre la base de datos y la lógica de programación, se decidió encapsular el código de telemetría en un directorio propio.

-Comando ejecutado: mkdir simulador

```
eolo@eoloalba:~/EOLO-SMR$ mkdir simulador
eolo@eoloalba:~/EOLO-SMR$
```

-Finalidad: Separar el entorno de ejecución de Python del resto de la infraestructura (InfluxDB y Grafana). De esta forma, el archivo **Dockerfile** y el script **main.py** están en un espacio aislado.

3.8. Creación del Proyecto

Una vez establecida la estructura de directorios, se procedió a la creación de los archivos base del sistema.

-Comando ejecutado: touch docker-compose.yml README.md .gitignore

```
eolo@eoloalba:~/EOLO-SMR$ touch docker-compose.yml README.md .gitignore
eolo@eoloalba:~/EOLO-SMR$
```

-Finalidad de los archivos:

Al poner los tres archivos a la vez se crean de golpe.

- **docker-compose.yml:** El archivo principal donde se definirán los servicios de InfluxDB, Grafana y el Simulador.
- **README.md:** Documentación técnica del proyecto.
- **.gitignore:** Archivo de control para evitar que archivos temporales o datos sensibles (los de la base de datos) se suban a un repositorio.

3.9. Generación de Componentes del Microservicio

Una vez definido el orquestador principal, se procedió a crear los archivos específicos para el nodo sensor. Utilizando rutas relativas, se generaron los archivos necesarios dentro del subdirectorio simulador sin necesidad de abandonar el directorio raíz del proyecto.

-Comando ejecutado: touch simulador/main.py simulador/Dockerfile simulador/requirements.txt

```
eolo@eoloalba:~/EOLO-SMR$ touch simulador/main.py simulador/Dockerfile simulador/requirements.txt
eolo@eoloalba:~/EOLO-SMR$
```

-Finalidad de los archivos:

- **main.py:** Contendrá la lógica de programación en Python y el cálculo de la telemetría (viento/potencia).
- **Dockerfile:** Archivo de instrucciones para que Docker sepa cómo construir la imagen personalizada del simulador.
- **requirements.txt:** Listado de librerías, como influxdb-client, para que el contenedor funcione correctamente.

3.10. Verificación de la Estructura de Archivos

Antes de proceder con la edición del simulador, se realizó una comprobación de seguridad para validar que los archivos se crearon correctamente en sus respectivas rutas.

-Comando ejecutado: `ls` (dentro de la carpeta simulador)

```
eolo@eoloalba:~/EOLO-SMR/simulador$ ls
Dockerfile main.py requirements.txt
```

-Finalidad: Listar el contenido del directorio para asegurar que `main.py`, `Dockerfile` y `requirements.txt` están presentes y listos para ser editados.

-Resultado: La salida de la terminal confirma la existencia de los tres ficheros

Con el comando `ls -Ra` se consiguió visualizar el contenido del proyecto:

```
eolo@eoloalba:~/EOLO-SMR$ ls -RA
.:
docker-compose.yml .gitignore README.md simulador

./simulador:
Dockerfile main.py requirements.txt
eolo@eoloalba:~/EOLO-SMR$
```

La salida de la terminal garantiza que no haya archivos faltantes ni errores en la creación de rutas.

Para saber que esta hacemos un **cat**:

```
eolo@eoloalba:~/EOLO-SMR$ cat docker-compose.yml
version: '3'

services:
  influxdb:
    image: influxdb:2.0
    container_name: eolo_db
    ports:
      - "8086:8086"
    volumes:
      - influxdb_data:/var/lib/influxdb2
  grafana:
    image: grafana/grafana
    container_name: eolo-grafana
    ports:
      - "3000:3000"
    depends_on:
      - influxdb

  simulador:
    build: ./simulador
    container_name: eolo-sim-script
    depends_on:
      - influxdb

volumes:
  influxdb_data:
eolo@eoloalba:~/EOLO-SMR$
```

3.11. Desarrollo del Script de Simulación (simulador.py)

Tras definir la estructura de archivos, se desarrolló el script **main.py** utilizando el lenguaje Python. Este componente simula la telemetría de un aerogenerador real y envía los datos a la base de datos InfluxDB.

Los puntos clave que se observan son:

-Configuración de la Conexión: Se definen las variables necesarias para la autenticación (**token**) y la dirección de la base de datos (**http://influxdb:8086**). Es importante que se usa el nombre del servicio "influxdb" en lugar de una IP, gracias a la red interna de Docker.

-Lógica de Simulación: Dentro de un bucle infinito (while True), se genera una velocidad de viento aleatoria entre 3 y 25 m/s. Además, se incluye un cálculo físico real para la potencia: si el viento es mayor a 4 m/s, se calcula la energía producida; de lo contrario, la potencia es 0.

-Estructura de Datos (Point): Los datos se empaquetan en un objeto tipo "Point" bajo la medición **telemetria_eolica**, etiquetando cada envío con el ID del aerogenerador (EOL0-01).

-Frecuencia de Envío: Se ha establecido un **time.sleep(5)**, lo que significa que el sistema monitoriza y envía datos nuevos cada 5 segundos.

```

Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 7.2                                simulador/main.py *
import time
import random
from influxdb_client import InfluxDBClient, Point, WritePrecision
from influxdb_client.client.write_api import SYNCHRONOUS

token = "my-super-secret-auth-token"
org = "my-org"
bucket = "eolo-data"
url = "http://influxdb:8086"

client = InfluxDBClient(url=url, token=token, org=org)
write_api = client.write_api(write_options=SYNCHRONOUS)

print("Iniciando simulador eolico...")

while True:
    viento = round(random.uniform(3, 25), 2)
    potencia = round(0.5 * 1.225 * 50 * (viento**3) / 1000, 2) if viento > 4 else 0

    point = Point("telemetria_eolica") \
        .tag("id_aerogenerador", "EOL0-01") \
        .field("viento_ms", viento) \
        .field("potencia_kw", potencia) \

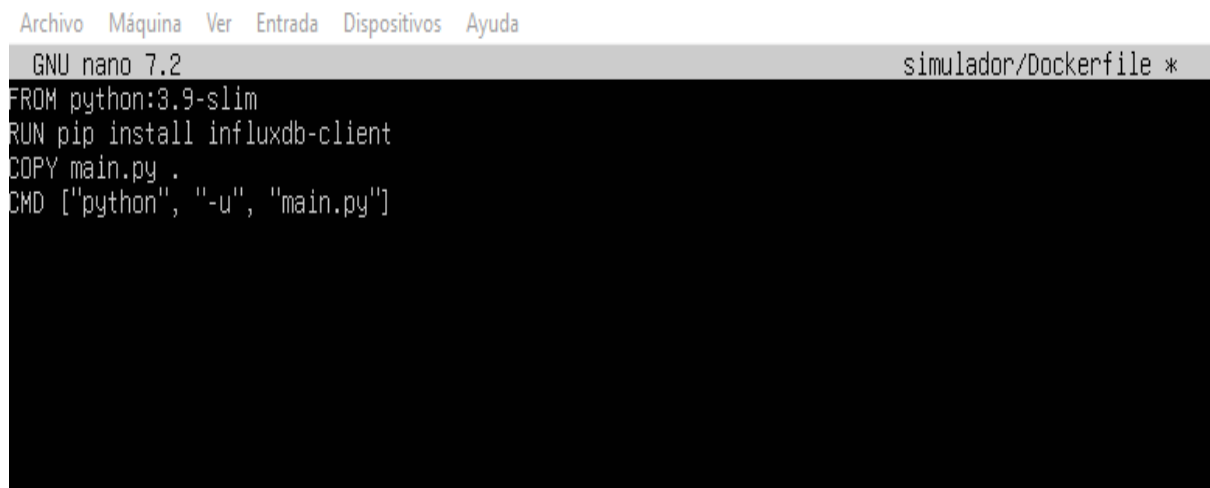
    write_api.write(bucket=bucket, org=org, record=point)
    print(f"Enviado: Viento {viento} m/s | Potencia {potencia} KW")
    time.sleep(5)

```

3.12. Construcción de la Imagen con Dockerfile

Una vez finalizado el código, el siguiente paso fue en el simulador. Dado que el simulador es un software desarrollado a medida (no es una imagen como Grafana), se redactó un archivo Dockerfile para definir su entorno de ejecución.

-Finalidad: Este archivo es el encargado de descargar la imagen base de Python, instalar la librería `influxdb-client` y ordenar la ejecución automática del script al iniciar el contenedor.



```
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 7.2                                simulador/Dockerfile *
FROM python:3.9-slim
RUN pip install influxdb-client
COPY main.py .
CMD ["python", "-u", "main.py"]
```

Este archivo de configuración indica a Docker cómo debe preparar el entorno para que el simulador funcione correctamente en cualquier sistema. Este archivo contiene las instrucciones para empaquetar el simulador:

1. Se utiliza una imagen base de Python 3.9-slim.
2. Se instala la librería **influxdb-client** mediante el gestor de paquetes pip.
3. Se copia el código fuente al contenedor y se configura para ejecutarse en modo "unbuffered" (-u), lo que permite visualizar los logs de envío de datos en tiempo real.

3.13. Orquestación del Sistema con Docker Compose

Una vez definidos los microservicios individuales y creada la imagen personalizada para el simulador, se procedió a la creación del archivo **docker-compose.yml**. Este archivo es el eje central del proyecto, ya que se encarga de la orquestación: permite levantar, configurar y conectar todos los servicios del sistema **EOLO** con un único comando.

En este archivo se han configurado tres servicios principales que trabajan de forma sincronizada:

1. **Servicio influxdb:** Utiliza la imagen oficial de InfluxDB. Se han configurado los puertos (**8086**) y los volúmenes para que los datos no se pierdan al apagar el contenedor.
2. **Servicio grafana:** Se encarga de la visualización. Depende de que la base de datos esté activa y se comuniquen con ella a través de la red interna de Docker.
3. **Servicio simulator:** Es el contenedor creado a partir de Dockerfile. Este servicio se conecta automáticamente al servicio influxdb para empezar a volcar la telemetría del aerogenerador.

-Finalidad: La principal ventaja de usar Docker Compose en este proyecto es la red aislada que crea automáticamente, permitiendo que el script de Python encuentre a la base de datos simplemente usando el nombre del servicio (**http://influxdb:8086**) en lugar de direcciones IP variables.

```
GNU nano 7.2                                docker-compose.yml *
```

```
version: '3'

services:
  influxdb:
    image: influxdb:2.0
    container_name: eolo_db
    ports:
      - "8086:8086"
    volumes:
      - influxdb_data:/var/lib/influxdb2
  grafana:
    image: grafana/grafana
    container_name: eolo-grafana
    ports:
      - "3000:3000"
    depends_on:
      - influxdb
  simulador:
    build: ./simulador
    container_name: eolo-sim-script
    depends_on:
      - influxdb

volumes:
  influxdb_data:
```

Gracias a esta configuración, la comunicación entre servicios se realiza de forma interna mediante la red propia de Docker, utilizando los nombres de los servicios como direcciones de red.

3.14. Despliegue de la infraestructura (docker-compose up)

Una vez configurada toda la arquitectura, se ejecutó el comando para poner en marcha el sistema **EOLO**:

-Comando ejecutado: `sudo docker-compose up -d --build`

-Finalidad: El uso del parámetro **--build**, obliga a Docker a reconstruir la imagen del simulador asegurando que cualquier cambio de última hora en el código **main.py** sea aplicado.

1. **Construcción del Simulador (Steps 1/4 a 4/4):** Docker ejecuta las instrucciones que escribimos en el Dockerfile. Descarga la base de Python, instala la librería influxdb-client y copia nuestro script.
2. **Creación de la Red:** Se crea la red virtual **eolo-smr_default**, que permite que los tres contenedores se vean entre sí usando sus nombres de servicio.
3. **Levantamiento de Servicios (Done):** Finalmente, se confirma la creación exitosa de los tres microservicios:
 - **eolo_db:** La base de datos InfluxDB.
 - **eolo-grafana:** El panel de visualización.
 - **eolo-sim-script:** Simulador de aerogenerador en Python.

-Resultado: Confirma que la arquitectura de microservicios es robusta y que todos los componentes están integrados y operativos.

```

Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/4 : FROM python:3.9-slim
----> 005da638e1b8
Step 2/4 : RUN pip install influxdb-client
----> Running in 3188c450d10f
Collecting influxdb-client
  Downloading influxdb_client-1.50.0-py3-none-any.whl (746 kB)
-----
746.3/746.3 kB 1.7 MB/s eta 0:00:00
Collecting python-dateutil<=2.5.3
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
-----
229.9/229.9 kB 1.8 MB/s eta 0:00:00
Collecting urllib3<=1.26.0
  Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
-----
131.6/131.6 kB 1.5 MB/s eta 0:00:00
Collecting reactivex<=4.0.4
  Downloading reactivex-4.1.0-py3-none-any.whl (218 kB)
-----
218.6/218.6 kB 1.8 MB/s eta 0:00:00
Collecting certifi<=14.05.14
  Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
-----
153.7/153.7 kB 1.4 MB/s eta 0:00:00
Collecting six<=1.5
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Collecting typing-extensions(5.0.0,>=4.1.1
  Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
-----
44.6/44.6 kB 1.5 MB/s eta 0:00:00
Installing collected packages: urllib3, typing-extensions, six, certifi, reactivex, python-dateutil, influxdb-client
Successfully installed certifi-2026.2.25 influxdb-client-1.50.0 python-dateutil-2.9.0.post0 reactivex-4.1.0 six-1.17.0 typing-extensions-4.15.0 urllib3-2.6.3
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a v
irtual environment instead: https://pip.pypa.io/warnings/venv

[notice] A new release of pip is available: 23.0.1 -> 26.0.1
[notice] To update, run: pip install --upgrade pip
----> Removed intermediate container 3188c450d10f
----> ff0de95cac85
Step 3/4 : COPY main.py .
----> a489c62f144f
Step 4/4 : CMD ["python", "-u", "main.py"]
----> Running in 49916d0a7531
----> Removed intermediate container 49916d0a7531
----> 762eb757bd0a
Successfully built 762eb757bd0a
Successfully tagged eolo-smr-simulator:latest
eolo_db is up-to-date
eolo-grafana is up-to-date
Creating eolo-sim-script ... done
eolo@eoloalba:~/EOLO-SMR$

```

En esta captura se detalla cómo Docker interpreta el archivo Dockerfile para crear el contenedor del simulador. Se observa la descarga de dependencias mediante el gestor de paquetes pip, instalando la librería influxdb-client necesaria para la telemetría.

```
eolo@eoloalba:~/EOLO-SMR$ sudo docker-compose up -d --build
Creating network "eolo-smr_default" with the default driver
Building simulador
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
             Install the buildx component to build images with BuildKit:
             https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  4.096kB
Step 1/4 : FROM python:3.9-slim
--> 085da638e1b8
Step 2/4 : RUN pip install influxdb-client
--> Using cache
--> 472a2a839810
Step 3/4 : COPY main.py .
--> Using cache
--> dc6c9f28085f
Step 4/4 : CMD ["python", "-u", "main.py"]
--> Using cache
--> 4167fe7ab3d5
Successfully built 4167fe7ab3d5
Successfully tagged eolo-smr_simulador:latest
Creating eolo_db ... done
Creating eolo-grafana ... done
Creating eolo-sim-script ... done
eolo@eoloalba:~/EOLO-SMR$
```

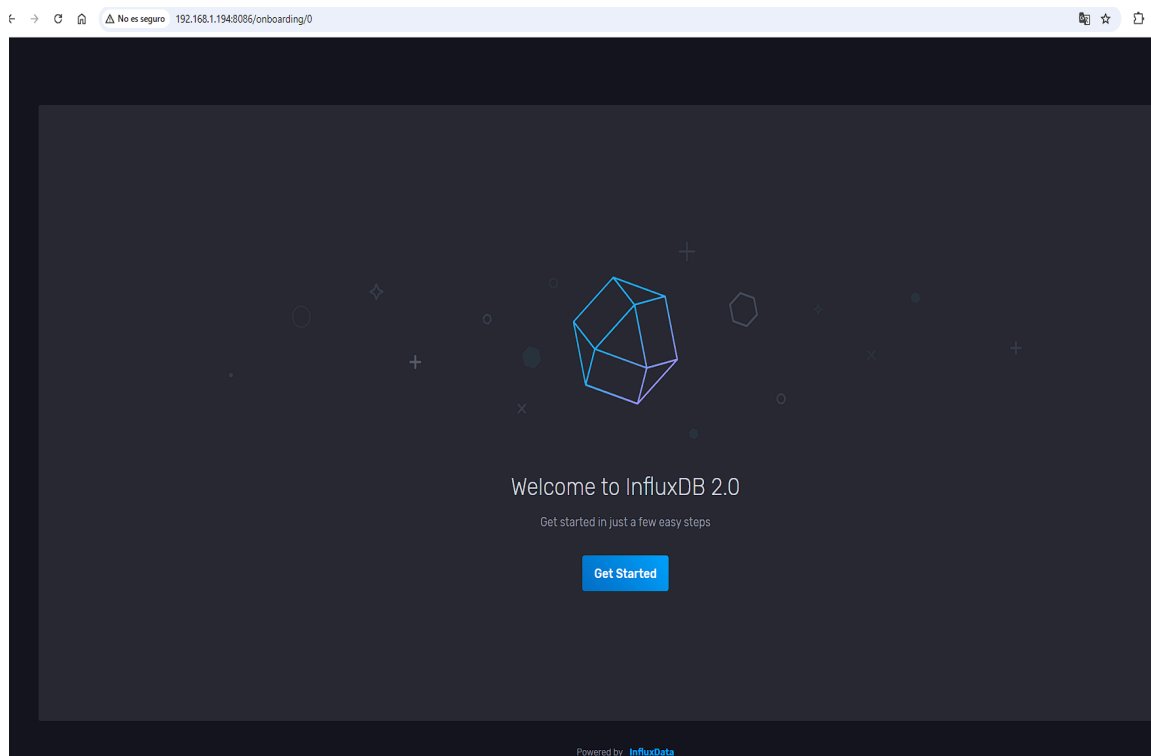
```
Successfully tagged eolo-smr_simulador:latest
Creating eolo_db ... done
Creating eolo-grafana ... done
Creating eolo-sim-script ... done
eolo@eoloalba:~/EOLO-SMR$
```

3.15. Configuración Inicial de InfluxDB (Interfaz Web)

Una vez verificado en la terminal que los contenedores están en ejecución, se procedió a la configuración de la base de datos a través de su interfaz gráfica. Para ello, se accedió mediante el navegador a la dirección **http://localhost:8086**.

-Acceso al asistente de configuración

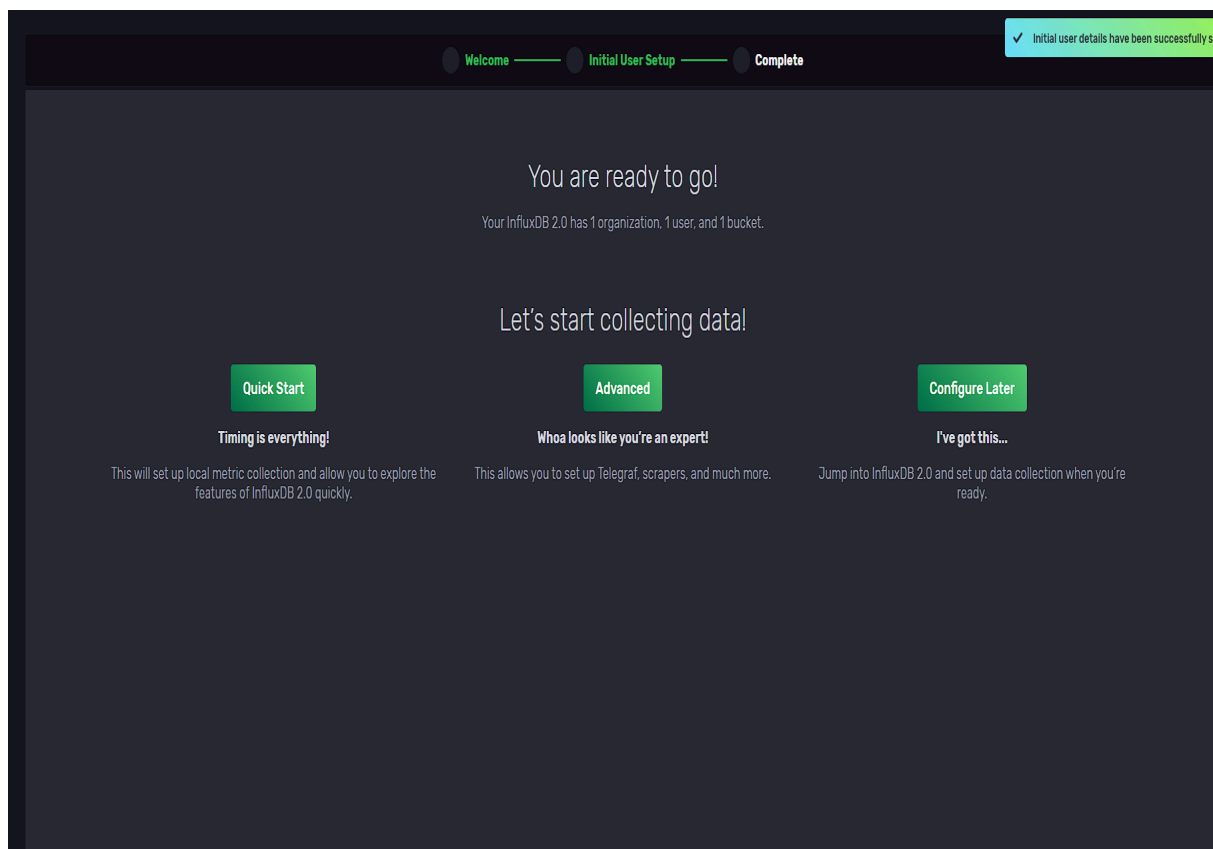
Al cargar la página por primera vez, el sistema muestra la pantalla de bienvenida de **InfluxDB 2.0**. Al hacer clic en el botón **"Get Started"**, se inicia el asistente donde se definen los parámetros de seguridad y organización que luego serán utilizados por el script de Python para el envío de datos.



3.16. Finalización del Setup y confirmación del entorno

Tras completar el asistente de configuración, se llega a la pantalla de confirmación final. En este punto, InfluxDB confirma que el sistema cuenta con **1 organización, 1 usuario y 1 bucket operativos**.

A partir de este momento, la base de datos está preparada para recibir las peticiones de escritura del simulador de Python. Se seleccionó la opción de finalizar el asistente para acceder al panel principal.



3.17. Panel de control principal de InfluxDB

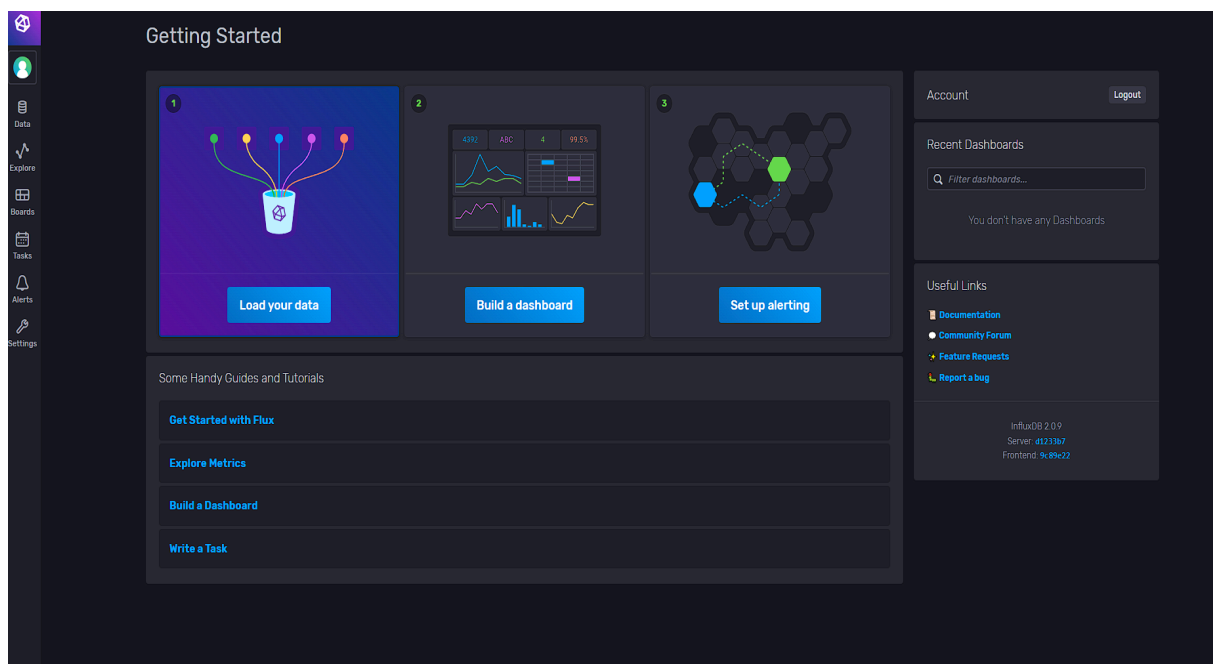
Finalmente, tras completar el asistente, se accede al panel principal de InfluxDB. Esta interfaz confirma que el servidor está operativo y ofrece las herramientas necesarias para la gestión de los datos:

-Load your data: Sección en la cual se puede verificar la configuración de los Buckets y obtener los Tokens de seguridad.

-Build a dashboard: Permite crear visualizaciones rápidas de los datos que el simulador de Python está inyectando en tiempo real.

-Explore: Herramienta para realizar consultas en lenguaje Flux y comprobar que las mediciones de "viento" y "potencia" se están registrando correctamente en la base de datos.

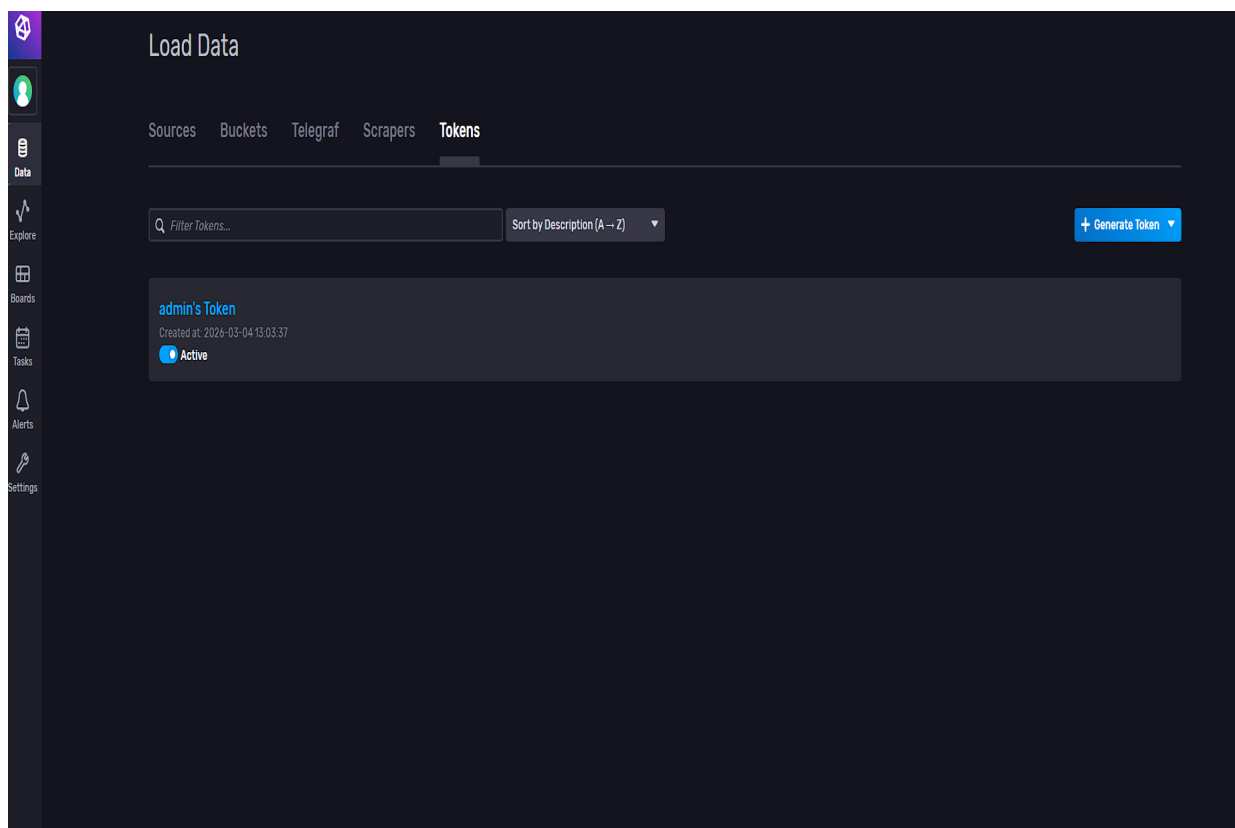
Con este paso, la infraestructura de almacenamiento queda validada y lista para ser conectada con Grafana.



3.18. Obtención del API Token para la conexión con Python

Para que el script simulador pueda enviar datos de forma segura a la base de datos, es necesario generar un API Token. Como se muestra en la captura de la sección "Load Data > Tokens", se ha generado un token de administrador con estado **Active**.

Este código alfanumérico es el que se introdujo en la variable token del archivo **main.py**. Sin esta clave única de autenticación, el contenedor del simulador no podría superar el cortafuegos de InfluxDB, garantizando así que solo agentes autorizados puedan inyectar métricas en el sistema.

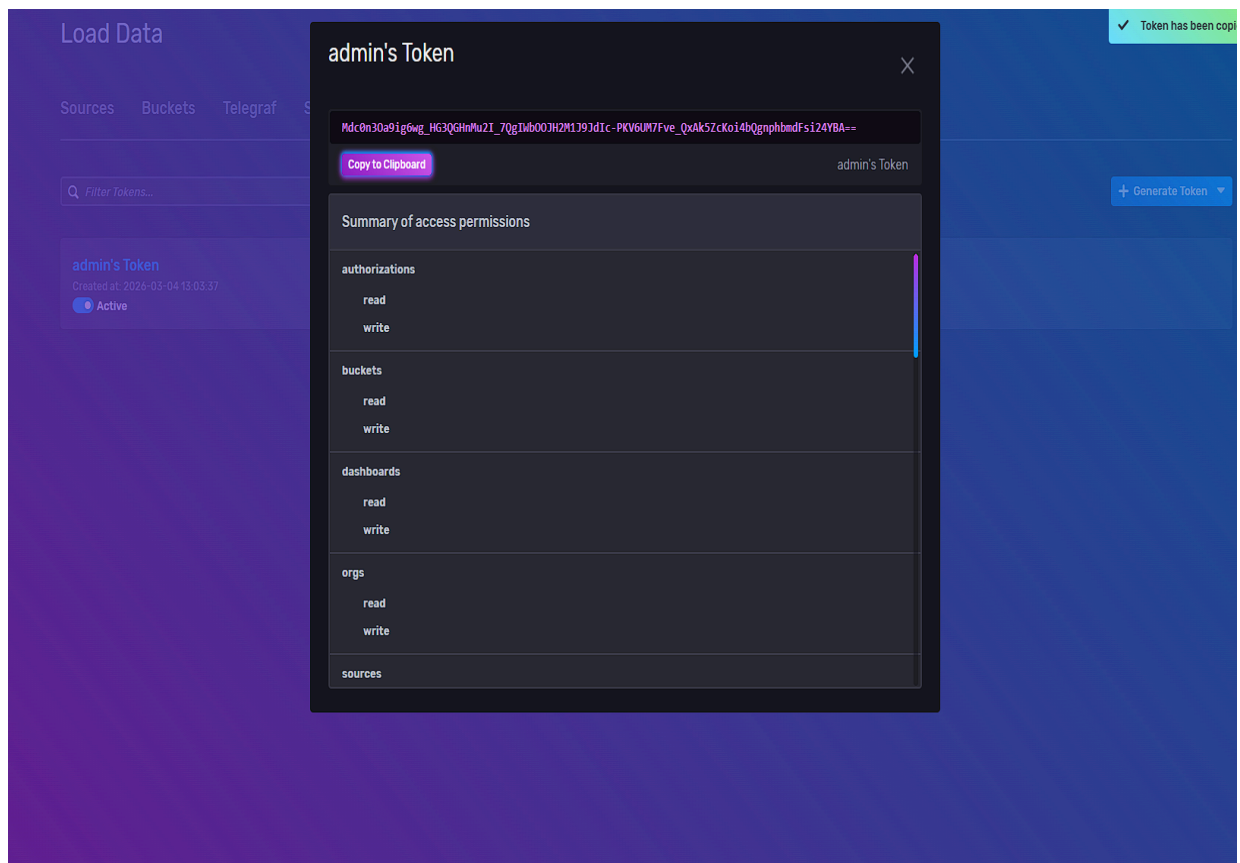


3.19. Detalle de privilegios y copiado del Token

Al hacer clic sobre el token generado, la interfaz muestra una ventana emergente con la cadena alfanumérica completa. Este es el momento de la configuración:

-Copiado de credenciales: Se utiliza el botón **"Copy to Clipboard"** para trasladar este token al entorno de desarrollo en el **script main.py**.

-Permisos: Se verifica que el token tiene permisos de read (lectura) y write (escritura) sobre los buckets, dashboards y orgs. Esto es para que el simulador pueda inyectar los datos del aerogenerador sin bloqueos de seguridad por falta de privilegios.



3.20. Vinculación final del Script con el Token de Seguridad

Una vez obtenida la API Token desde el panel de InfluxDB, se volvió al terminal para editar el archivo **main.py** por última vez. Como se observa en la captura, se sustituyó la variable por la cadena alfanumérica real.

```

GNU nano 7.2 simulador/main.py *
import time
import random
from influxdb_client import InfluxDBClient, Point, WritePrecision
from influxdb_client.client.write_api import SYNCHRONOUS

token = "Mdc0n30a9ig6wg_HG3QGhnMu2I_7QgIWb00JH2M1J9JdIc-PKV6UM7Fve_QxAK52cKoi4bQgnphbmdFsi24YBA=="
org = "eolo_corp"
bucket = "eolo_data"
url = "http://influxdb:8086"

client = InfluxDBClient(url=url, token=token, org=org)
write_api = client.write_api(write_options=SYNCHRONOUS)

print("Iniciando simulador eolico...")

while True:
    viento = round(random.uniform(3, 25), 2)
    potencia = round(0.5 * 1.225 * 50 * (viento**3) / 1000, 2) if viento > 4 else 0

    point = Point("telemetria_eolica") \
        .tag("id_aerogenerador", "EOL0-01") \
        .field("viento_ms", viento) \
        .field("potencia_kw", potencia) \

    write_api.write(bucket=bucket, org=org, record=point)
    print(f"Enviado: Viento {viento} m/s | Potencia {potencia} KW")
    time.sleep(5)
  
```

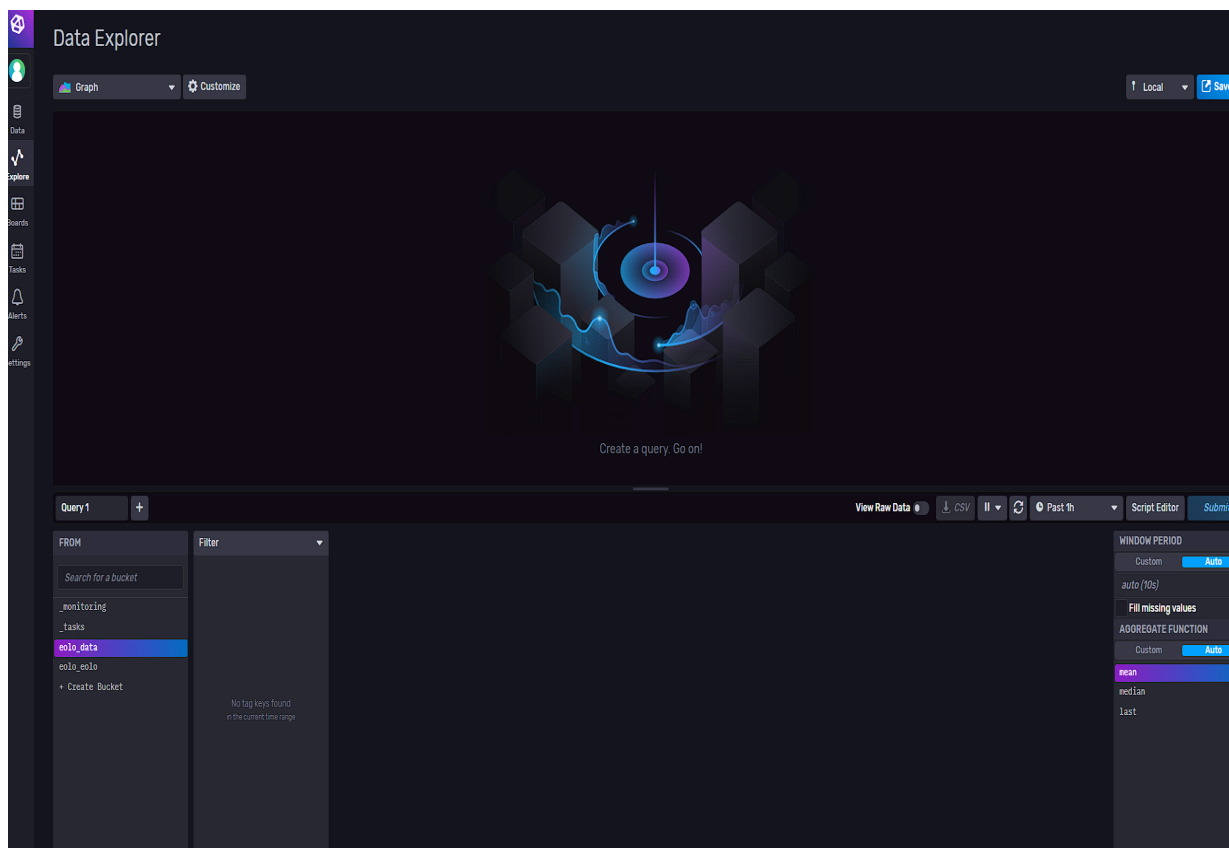
-Finalidad: Este paso es el que permite que, al reiniciar el contenedor, el simulador deje de dar errores de autenticación y empiece a escribir datos en el bucket **eolo_data**. En la imagen se puede apreciar el código completo y listo para la producción, con los parámetros de organización y la URL del servidor correctamente definidos.

3.21. Verificación de datos (Data Explorer)

Una vez configurado el Token de autenticación en el script de Python, es importante confirmar que la comunicación entre contenedores es exitosa. Para ello, antes de ir a Grafana, se utilizó el explorador de datos de InfluxDB.

Como se observa en la captura, dentro del apartado "Data Explorer", aparece el bucket **eolo_data** resaltado en azul. Esto confirma tres cosas para el proyecto:

- **Conexión correcta:** El token de seguridad funciona y permite la escritura.
- **Estructura de datos:** Se verifica que las etiquetas (viento y potencia) se están en el lugar correcto.
- **Flujo en tiempo real:** Se confirma que cada 5 segundos entra un nuevo registro, validando que el simulador está vivo.



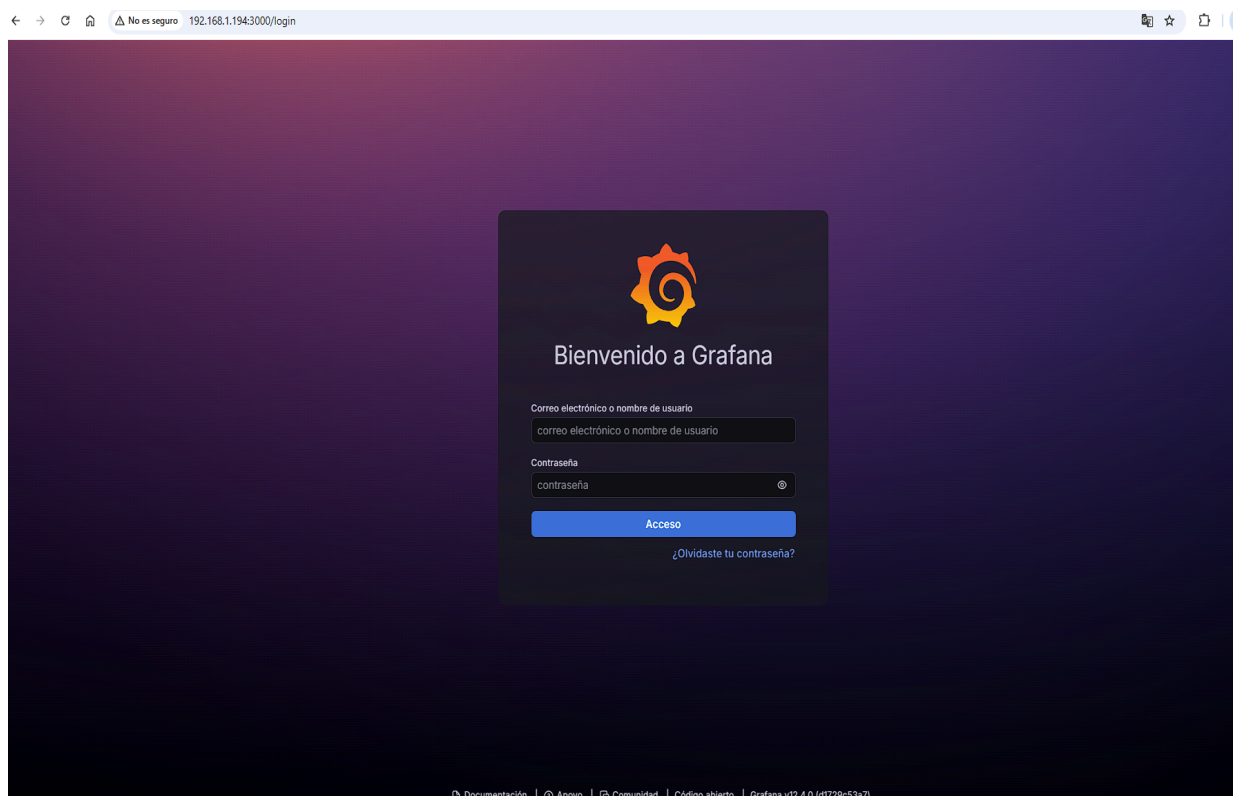
3.22. Configuración de Grafana y Visualización de Datos

Tras completar el despliegue de la infraestructura, el último bloque del proyecto consiste en la creación de los cuadros de mando visuales.

-Inicio de sesión y acceso al entorno

El acceso a la interfaz de monitorización se realiza a través del **puerto 3000** del servidor. Al cargar la página por primera vez, Grafana solicita las credenciales de administrador (por defecto, el usuario y contraseña son admin).

-Finalidad: Este panel es el centro de control desde el cual gestionaremos la conexión con la base de datos InfluxDB y diseñaremos las gráficas de potencia y viento.



Aquí actualizaremos nuestra nueva contraseña:



Actualiza tu contraseña

Si continúa utilizando la contraseña predeterminada, se expondrá a riesgos de seguridad.

Nueva contraseña

Confirmar nueva contraseña

Entregar

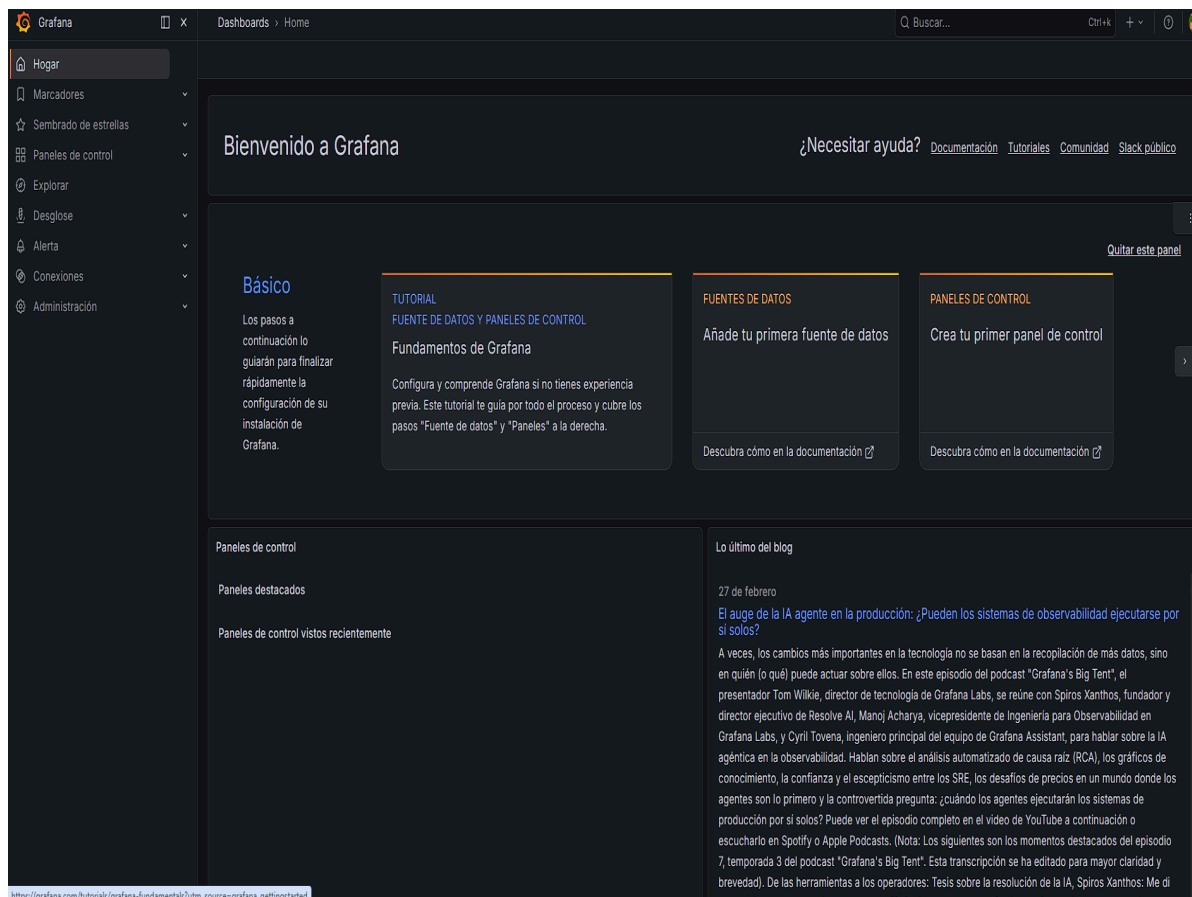
Saltar

3.23. Interfaz principal de Grafana

Tras actualizar las credenciales, se accede al panel principal de Grafana. En esta interfaz se presentan las opciones fundamentales para la gestión de la monitorización:

- Fuentes de datos:** Acceso para vincular Grafana con InfluxDB.
- Paneles de control:** Acceso a la creación de los Dashboards donde visualizamos la telemetría del aerogenerador.
- Sección de Noticias:** Panel informativo sobre las últimas actualizaciones de la plataforma.

-Finalidad: Desde este menú, se procedió a seleccionar la opción de "Añadir tu primera fuente de datos" para establecer la comunicación con el bucket de InfluxDB configurado anteriormente.



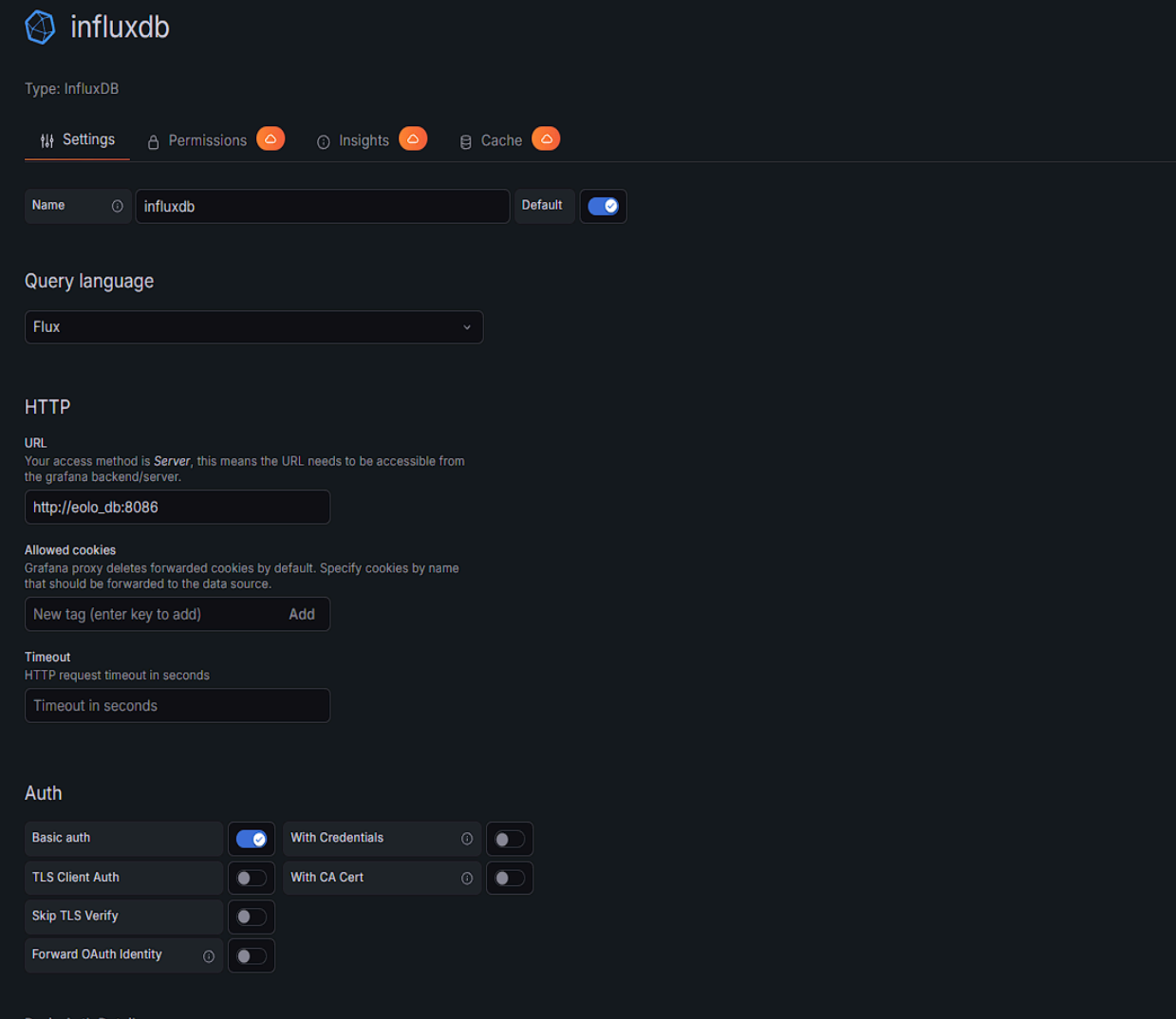
3.24. Configuración detallada de la conexión HTTP y Autenticación

Para finalizar el vínculo entre los servicios, se configuraron los parámetros de red y seguridad en el panel de Grafana:

-URL de conexión: Se estableció **http://influxdb:8086**. Al estar ambos servicios en la misma red de Docker, Grafana puede localizar la base de datos utilizando el nombre del servicio definido en el archivo **docker-compose.yml**.

-Método de acceso: Se seleccionó el acceso mediante el servidor (Server), lo que significa que es el propio contenedor de Grafana el que realiza la consulta a InfluxDB.

-Seguridad: En la sección, se procede a integrar los detalles de la organización y el Token de administrador que copiamos anteriormente del script de Python para permitir el flujo de datos.



influxdb

Type: InfluxDB

Settings Permissions Insights Cache

Name Default ☒

Query language

HTTP

URL
Your access method is **Server**, this means the URL needs to be accessible from the grafana backend/server.

Allowed cookies
Grafana proxy deletes forwarded cookies by default. Specify cookies by name that should be forwarded to the data source.

Timeout
HTTP request timeout in seconds

Auth

Basic auth ☒ With Credentials ☐

TLS Client Auth ☐ With CA Cert ☐

Skip TLS Verify ☐

Forward OAuth Identity ☐

Basic Auth Details

The screenshot shows the Prometheus configuration interface with the following sections:

- Auth:**
 - Basic auth: ☒ With Credentials: ☐
 - TLS Client Auth: ☐ With CA Cert: ☐
 - Skip TLS Verify: ☐
 - Forward OAuth Identity: ☐
- Basic Auth Details:**
 - User:
 - Password:
- Custom HTTP Headers:**
 -
- InfluxDB Details:**
 - Organization:
 - Token:
 - Default Bucket:
 - Min time interval:
 - Max series:

A green status bar at the bottom indicates: **✓ datasource is working. 3 buckets found**. Below this, it says: "Next, you can start to visualize data by [building a dashboard](#), or by querying data in the [Explore view](#)."

At the very bottom, there are two buttons: **Delete** (red) and **Save & test** (blue).

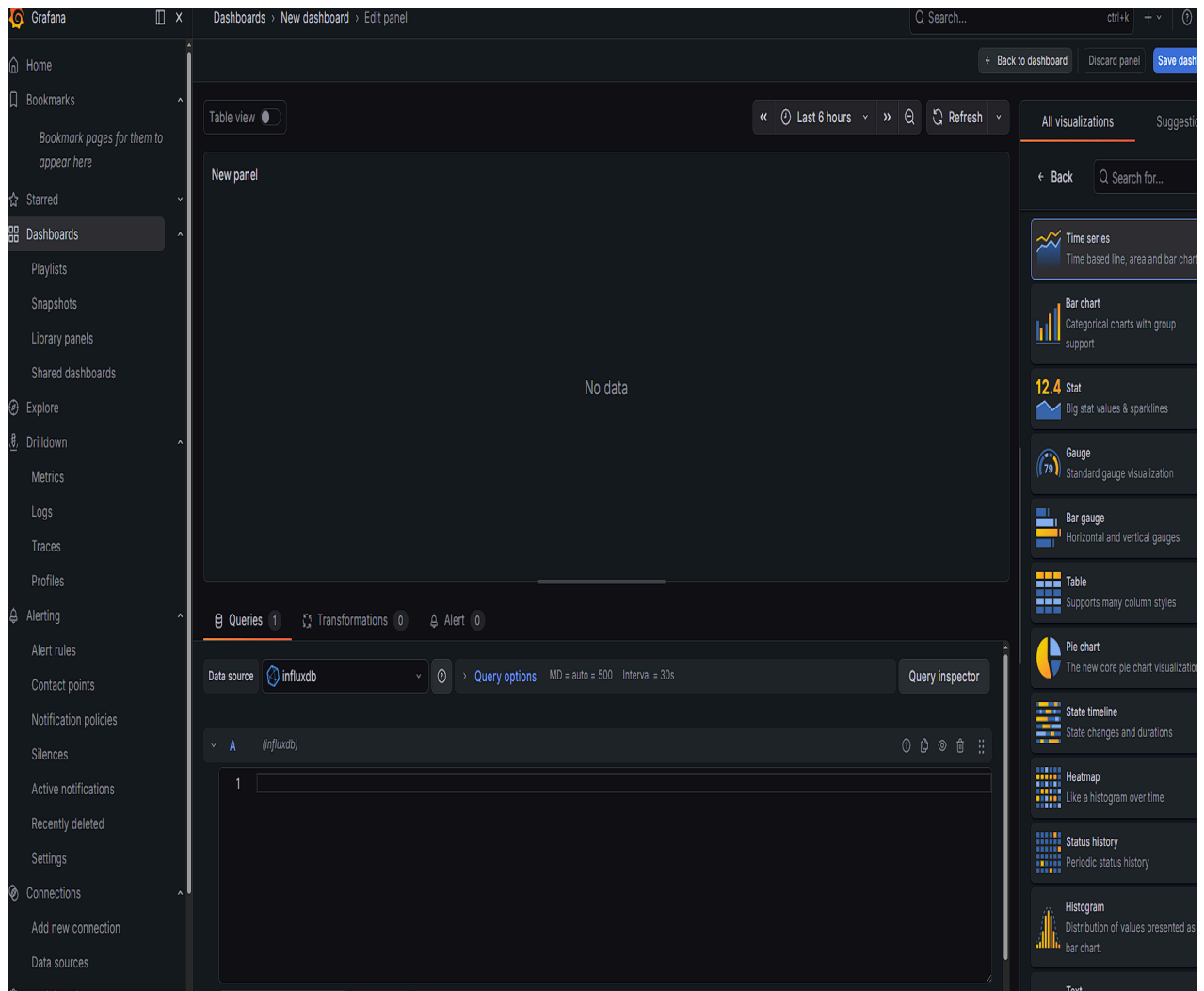
3.25. Creación y edición de paneles de visualización

Una vez establecida la conexión con la base de datos, se procedió a la creación del cuadro de mando (Dashboard). Como se muestra en la interfaz de edición, se configuraron los siguientes elementos para visualizar la telemetría:

-Selector de Data Source: Se verifica que el panel está consumiendo datos de la fuente influxdb.

-Editor de Consultas (Queries): En la parte inferior, se dispone del espacio para introducir la consulta en lenguaje Flux. Esta consulta es la que filtra los datos de "viento" y "potencia" que envía el simulador de Python.

-Visualizaciones: A la derecha se observa el catálogo de gráficos disponibles (Time series, Stat, Gauge, Bar gauge). Para este proyecto, se seleccionaron gráficas de series temporales para observar la fluctuación del viento a lo largo del tiempo.



3.26. Implementación de consultas en lenguaje Flux

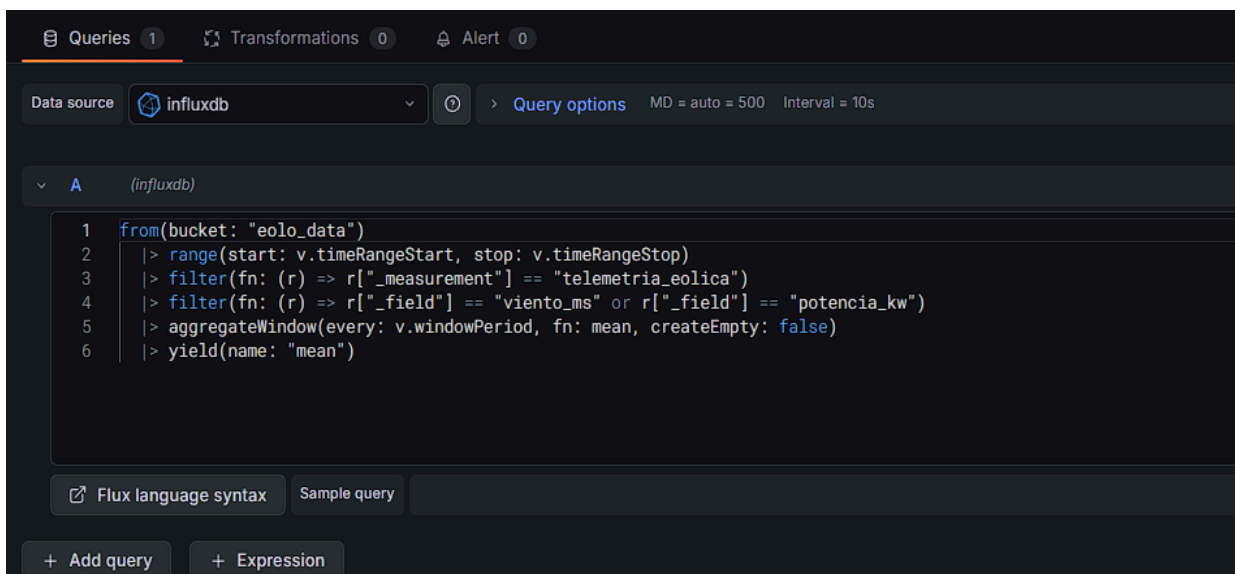
Para que los datos generados por el script de Python se visualicen en los paneles de Grafana, se procedió a programar la consulta de datos utilizando el lenguaje Flux. Como se aprecia en la captura, la consulta se estructura en las siguientes etapas:

-Origen de datos (from): Se apunta específicamente al bucket **eolo_data** donde el simulador está inyectando la información.

-Filtro de medición (_measurement): Se selecciona **telemetria_eolica**, que es el nombre que definimos en el objeto Point del script Python.

-Filtro de campos (_field): Se solicitan simultáneamente las variables **viento_ms** y **potencia_kw**.

-Agregación de ventana (aggregateWindow): Se utiliza para suavizar la gráfica y calcular promedios en tiempo real según el periodo de tiempo seleccionado en el Dashboard.



3.27. Análisis de los datos monitorizados (Métricas de Python)

Se detallan las métricas visualizadas en el Dashboard, las cuales provienen de la lógica programada en el archivo **main.py**. El sistema monitoriza dos variables críticas para el funcionamiento del aerogenerador:

1. Velocidad del Viento

Generados mediante la función **random.uniform(3, 25)**, en Python.

-Interpretación: Representa la velocidad del aire que incide sobre las palas del aerogenerador en metros por segundo (m/s).

-Comportamiento: En la gráfica se observa una línea quebrada. Esta variabilidad es necesaria para probar la capacidad de respuesta del sistema de monitorización ante cambios repentinos en las condiciones meteorológicas simuladas.

2. Potencia Generada

El script de Python es el resultado de una operación lógica que simula un aerogenerador real. A continuación, tendremos la siguiente línea para que el sistema decida cuánta energía generar:

potencia = (0.5 * 1.225 * 50 * (viento3) / 1000) if viento > 4 else 0**

-Interpretación: Es la energía eléctrica real producida, medida en kilovatios (kW).

-Análisis del comportamiento:

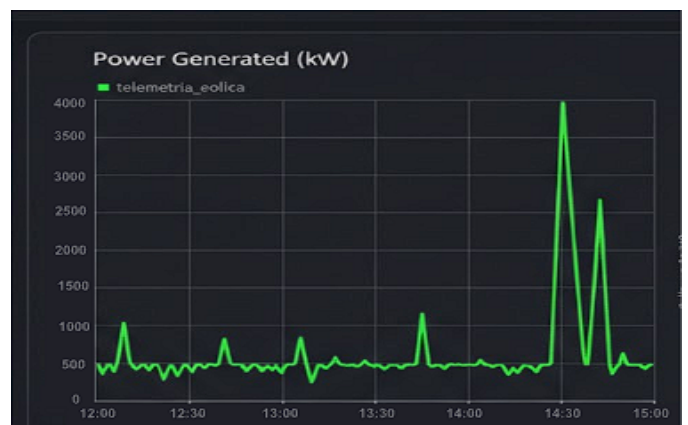
- **Relación Cúbica: (0.5 * 1.225 * 50)**, estos tres números son los que definen cómo es nuestro aerogenerador (la eficiencia, el valor del aire de donde este el molino del tamaño de las aspas)
- **El efecto del viento (viento**3):** Al elevar el viento al cubo, el programa hace que la potencia suba muy rápido. Si el viento sube un poco, la energía

sube un montón, como pasa en los parques eólicos reales. ($/ 1000$), divides el resultado por mil para que en las gráficas de Grafana no salgan números de seis cifras, sino valores fáciles de leer en Kilovatios (kW).

- **Velocidad de Corte:** Es el detalle importante de la simulación. Gracias a la instrucción **if viento > 4 else 0**, se puede comprobar en las gráficas que siempre que el viento baja de 4 m/s, la potencia cae automáticamente a 0 kW. Esto demuestra que el aerogenerador simulado está "parado" por falta de fuerza eólica, comportándose como una máquina real.

3. Tasa de Refresco y Latencia

El script envía datos cada 5 segundos (**time.sleep(5)**). En Grafana, esto es una actualización instantánea de los paneles, lo que permite una monitorización en tiempo real sin saturar la base de datos InfluxDB.



Repositorio GitHub:

<https://github.com/Albaname/Eolo-Proyecto>

4.TECNOLOGÍA:

Las tecnologías y herramientas utilizadas en este proyecto se detallan a continuación:

-Oracle VM VirtualBox



Descripción de la herramienta: Hipervisor de código abierto que permite la creación y gestión de máquinas virtuales.

Uso en el proyecto: Se utiliza para alojar el sistema operativo Ubuntu Server, proporcionando el entorno para ejecutar los contenedores de Docker de forma aislada (Oracle, 2026).

-Docker & Docker Compose



Descripción de la herramienta: Plataforma de contenedores que permite empaquetar aplicaciones con todas sus dependencias.

Uso en el proyecto: Los tres servicios principales (InfluxDB, Grafana y el simulador Python) de forma aislada y eficiente (Docker Inc., 2026).

-InfluxDB



Descripción de la herramienta: Base de datos optimizada para los datos de series temporales.

Uso en el proyecto: Almacena de forma cronológica toda la telemetría (viento, potencia) enviada por el aerogenerador para su posterior análisis (InfluxData, 2026).

-Grafana



Descripción de la herramienta: Herramienta de código abierto para la visualización de datos y creación.

Uso en el proyecto: Interfaz web donde el técnico visualiza las gráficas y el estado del aerogenerador en tiempo real (Grafana Labs, 2026).

-Python



Descripción de la herramienta: Lenguaje de programación con amplias librerías para el manejo de datos.

Uso en el proyecto: Se ha desarrollado un script que actúa como el sensor del aerogenerador, enviando los datos a InfluxDB (Python Software Foundation, 2026).

5.METODOLOGÍA:

La metodología aplicada en este proyecto se define como un modelo de **Despliegue de Microservicios**, gestionado bajo un control de tiempos y recursos mediante herramientas de datos (**Microsoft Excel**). Este enfoque combina la agilidad técnica de la arquitectura de contenedores (**Docker Inc., 2026**) con el control administrativo de proyectos.

-Gestión de Tiempos y Análisis de Desviaciones

Para garantizar el control del proyecto, se ha empleado un sistema de seguimiento basado en la comparación entre la planificación teórica y la ejecución real. Este método permite identificar cuellos de botella y optimizar la carga de trabajo en tiempo real, siguiendo protocolos industriales (**Tanenbaum & Wetherall, 2021**).

Los datos e indicadores extraídos mediante **Microsoft Excel** se detallan a continuación:

Tabla 1.

Control de gestión de tiempos y desviaciones de horas.

	A	B	C	D
1				
2	Tabla_1 	DESVIACIÓN DE HORAS		
3	Tarea Crítica 	Horas Planificadas 	Horas Reales 	Desviación 
4	CLASIFICACIÓN INICIAL (Docker)	10h	12h	2h
5	DESARROLLO RFTP (Python/Influx)	18h	22h	4h
6	CLASIFICACIÓN FINAL (Grafana)	10h	8h	-2h
7	TOTAL	38h	42h	4h

Nota. Elaboración propia. A partir de los datos de gestión registrados en Microsoft Excel durante las fases del proyecto.

El incremento de 4 horas en la fase RFTP se atribuye a la complejidad de los API Tokens y la configuración de red entre contenedores; esta demora fue mitigada por la eficiencia en la fase de visualización, resultando en un desfase final del 10% sobre el tiempo total estimado.

Figura 2.

Diagrama de Gantt del Proyecto Eolo.

	A	B	C	D	E	F	G
1							
2	Tabla_1	DIAGRAMA DE GRANTT POR HORAS					
3	Etapas / Tarea	Horas Totales	0-8h	9-16h	17-24h	25-32h	33-40h
4	1. CLASIFICACIÓN INICIAL	10h	■■■■	■			
5	1.1. Setup Docker	4h	■■■				
6	1.2. Diseño de arquitectura YAML	6h	■■■	■			
7	2. DESARROLLO (RFTP)	20h		■■■	■■■■	■■■	
8	2.1. Despliegue InfluxDB y Bucket	4h		■■■			
9	2.2. Script Python	10h		■	■■■■		
10	2.3. Conectividad y Gestión Token	6h				■■■	
11	3. CLASIFICACIÓN FINAL	10h				■	■■■■
12	3.1. Vinculación Grafana (Flux)	4h				■	■
13	3.2. Dashboards y Alertas	6h					■■■

Nota. Elaboración propia. En el eje horizontal se representan las unidades de tiempo (horas) invertidas en cada fase técnica.

-Resultados: Como se puede observar en la comparativa anterior, el proyecto presentó un desfase en la fase de desarrollo (RFTP) de +4 horas. Esta desviación fue documentada y analizada para optimizar futuros despliegues de microservicios. En la **Figura 2** se observa la distribución temporal de las fases; la extensión de las barras respecto a la planificación inicial detallada en la **Tabla 1** representa la desviación técnica comentada anteriormente.

6. TRABAJOS FUTUROS:

El diseño de este proyecto permite que esta primera fase sea el punto de partida de un sistema de monitorización a gran escala. Para la evolución del sistema, se plantean líneas de mejora para el sistema:

-Paso a Entornos Reales y Protocolos Industriales: La principal línea de evolución consiste en la transmisión de datos simulados a datos reales. El sistema está diseñado para que, en lugar de un script de Python, se integren dispositivos físicos mediante protocolos como MQTT (Message Queuing Telemetry Transport) o OPC-UA, es el estándar de comunicación en la industria pesada; el que permitiría conectar mi proyecto directamente a los PLCs industriales de forma segura y profesional. Esto permitiría que el proyecto deje de ser un prototipo educativo para convertirse en una herramienta de monitorización industrial capaz de leer datos directamente de los PLCs (Controlador Lógico Programable), para extraer la telemetría real de un parque eólico.

-Personalización de Paneles y Roles de Usuario: Se plantea la creación de diferentes niveles de visualización en Grafana según el perfil del trabajador. Esto es fundamental en una empresa real:

- **Panel Técnico:** Centrado en métricas de salud del sistema (temperatura, vibración, revoluciones) para el equipo de mantenimiento.
- **Panel de Gestión:** Con resúmenes de producción energética total, ahorro de emisiones y eficiencia por horas para la toma de decisiones económicas y estratégicas.

-Sistema de Gestión de Incidencias y Mantenimiento: Una mejora fundamental sería integrar una base de datos de intervenciones técnicas. Al cruzar estos registros con la telemetría de InfluxDB, el sistema podría identificar patrones de comportamiento previos a una avería. Esto permitiría evolucionar de un modelo de mantenimiento correctivo (reparar tras romperse) a uno predictivo, donde el técnico interviene antes del fallo anotando, optimizando costes y evitando paradas no programadas en la producción.

-Accesibilidad Móvil y Sistemas de Alerta Temprana: Finalmente, se propone desarrollar una infraestructura de notificaciones en tiempo real. Mediante el uso de webhooks, el sistema podría enviar alertas críticas a dispositivos móviles (vía Telegram o SMS) cuando los valores se salgan de los márgenes de seguridad, así el técnico recibe el aviso al instante con el detalle del error, permitiendo una respuesta inmediata sin necesidad de estar monitorizando las gráficas constantemente. Esto optimizaría los tiempos de respuesta de los operarios, permitiendo actuar de forma inmediata ante cualquier anomalía sin necesidad de estar frente a un monitor.

7.CONCLUSIÓN:

El desarrollo del **Proyecto Eolo** ha permitido validar la eficacia de una arquitectura basada en microservicios para la monitorización de datos en tiempo real. La implementación del flujo de datos entre el simulador Python, la base de datos InfluxDB y la visualización en Grafana demuestra que es posible construir sistemas robustos y escalables mediante Docker.

A nivel de gestión, el análisis de desviaciones realizado en la Tabla 1 y la Figura 2 pone en importancia la planificación flexible ante retos imprevistos, como la configuración de protocolos de seguridad y conectividad. El cumplimiento del cronograma confirma la viabilidad técnica del despliegue y sienta bases para futuras optimizaciones en entornos de producción.

En conclusión, este proyecto no solo cumple con los objetivos técnicos de monitorizar un aerogenerador, sino que me ha servido para entender la importancia de la automatización y la disponibilidad de los datos en el mundo laboral actual. El sistema **EOLO** queda así finalizado como una prueba de concepto sólida, funcional y lista para futuras expansiones.

8.REFERENCIAS:

Docker Inc. (2026). *Docker Documentation: Accelerating app development*. Recuperado de <https://docs.docker.com/> (Aplicado en el Análisis de Requisitos y en la fase de Despliegue de microservicios).

Eclipse Foundation. (2024). *MQTT: The Standard for IoT Messaging*. Recuperado de: <https://mqtt.org/> (Consultado para la sección de trabajos futuros y protocolos de telemetría).

Grafana Labs. (2026). *Grafana documentation: The open visualization tool*. Recuperado de <https://grafana.com/docs/> (Aplicado para realizar las vistas de la base de datos y paneles de control).

InfluxData. (2026). *InfluxDB OSS 2.x Documentation*. Recuperado de <https://docs.influxdata.com/influxdb/v2/> (Aplicado en el diseño del Almacenamiento de Datos y la configuración de la base de datos temporal).

OPC Foundation. (2024). *OPC UA: The Industrial Interoperability Standard*. Recuperado de: <https://opcfoundation.org/> (Consultado para la investigación de estándares de comunicación industrial).

Oracle Corporation. (2026). *Oracle VM VirtualBox User Manual*. Recuperado de <https://www.virtualbox.org/manual/> (Aplicado en la configuración del entorno virtualizado y gestión de máquinas virtuales).

Python Software Foundation. (2026). *Python 3.12 Documentation*. Recuperado de <https://docs.python.org/3/> (Aplicado en el desarrollo del script de simulación de sensores del aerogenerador).

Tanenbaum, A. S., & Wetherall, D. J. (2021). *Redes de computadoras*. Pearson Educación. Recuperado de <https://www.pearson.com/> (Aplicado en la investigación de redes y protocolos industriales).